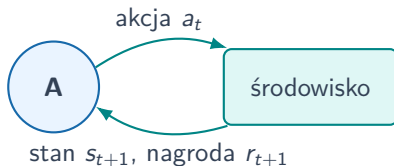


Uczenie ze wzmocnieniem na przykładzie FrozenLake

Od MDP i programowania dynamicznego do Q-Learning, DQN i akcji ciągłych

Grzegorz Madejski

Informatyka / Inteligencja obliczeniowa



Mapa wykładu: od problemu do algorytmu

- 1 Kiedy problem jest problemem decyzyjnym?
- 2 Jak zapisać go jako **MDP** (*Markov Decision Process*)?
- 3 Co można zrobić, gdy znamy model? **DP** (*Dynamic Programming*)
- 4 Co robić, gdy modelu nie znamy? **RL** (*Reinforcement Learning*)
- 5 Jak działa **Q-Learning** (*model-free, off-policy*)?
- 6 Kiedy potrzebujemy sieci: **DQN** (*Deep Q-Network*)?

Cel dydaktyczny

Po wykładzie student powinien umieć dobrać rodzinę metod do problemu:

- znany model vs uczenie z doświadczenia,
- stany i akcje dyskretne vs ciągłe,
- tablica wartości vs aproksymacja siecią.

Kiedy RL jest sensowne?

Cechy problemu, które sprzyjają RL

- decyzje tworzą sekwencję w czasie: $s_t \rightarrow a_t \rightarrow s_{t+1}$,
- decyzja teraz wpływa na możliwości za kilka kroków,
- nagroda jest opóźniona (*delayed reward*),
- nie mamy etykiet typu: „w tym stanie poprawna akcja to X”,
- można eksperymentować w symulatorze lub bezpiecznym środowisku.

Uwaga

RL zwykle nie jest pierwszym wyborem, gdy problem jest jednorazowy, bardzo niebezpieczny albo tani do rozwiązania klasycznego.

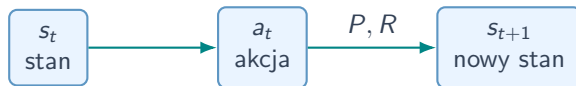


Przykłady: gry, robotyka, rekomendacje sekwencyjne, zarządzanie zasobami, sterowanie procesem.

MDP jako pięćka

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$$

- $s \in \mathcal{S}$ - **stan** (*state*), opis sytuacji.
- $a \in \mathcal{A}(s)$ - **akcja** (*action*), możliwa decyzja.
- $P(s' | s, a)$ - **model przejść** (*transition model*).
- $R(s, a, s')$ - **nagroda** (*reward*), czasem kara (*penalty*).
- $\gamma \in [0, 1]$ - **dyskont** (*discount factor*).



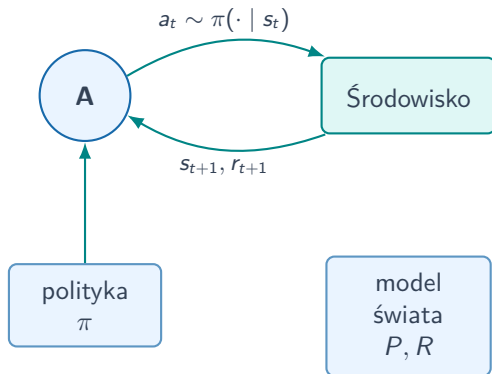
własność Markowa:

$$Pr(s_{t+1} | s_t, a_t, \dots) = Pr(s_{t+1} | s_t, a_t)$$

Polityka (*policy*)

$$\pi(a | s) = \Pr(A_t = a | S_t = s)$$

- deterministyczna: $\mu(s) = a$,
- stochastyczna: rozkład prawdopodobieństwa po akcjach,
- optymalna: π^* maksymalizuje oczekiwany zwrot.



Nie mylić

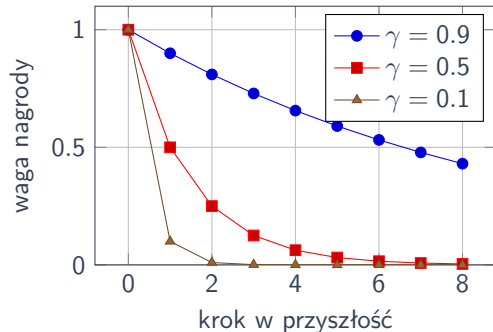
Model $P(s' | s, a)$ mówi, co zrobi środowisko. Polityka $\pi(a | s)$ mówi, co zrobi agent.

Zwrot, horyzont i dyskonto

Zwrot (*return*)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- $\gamma \approx 0$ - agent krótkowzroczny.
- $\gamma \approx 1$ - agent dalekowzroczny.
- Dyskonto stabilizuje nieskończone trajektorie.



Funkcje wartości: stan i akcja

Wartość stanu (*state-value function*)

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Mierzy, jak „dobrze” jest być w stanie s , gdy dalej działamy według π .

Wartość akcji (*action-value function*)

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Mierzy opłacalność rozpoczęcia od konkretnej akcji a w stanie s .

Zależność

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q^{\pi}(s, a)$$

Ewaluacja polityki

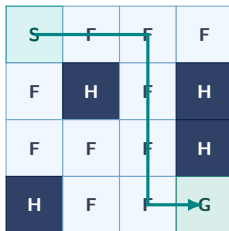
$$V^{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) (R(s, a, s') + \gamma V^{\pi}(s'))$$

Optymalność

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) (R(s, a, s') + \gamma V^*(s'))$$

- DP rozwiązuje te równania, gdy P i R są znane.
- RL próbuje je przybliżać z próbek doświadczenia (s, a, r, s') .

FrozenLake jako MDP



- Stan: numer pola $s = i \cdot n + j$.
- Akcje: lewo, dół, prawo, góra.
- Nagroda: zwykle 1 za wejście do celu, 0 gdzie indziej.
- Stany terminalne: dziura (*hole*) i cel (*goal*).
- Wariant `is_slippery=True`: akcja może skończyć się ruchem w innym kierunku.

Trudność dydaktyczna

Nagroda jest rzadka (*sparse reward*): większość kroków daje 0, więc agent musi odkryć długą sekwencję sensownych akcji.

Gymnasium API: minimalny kontrakt ze środowiskiem

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=True)
obs, info = env.reset(seed=42)
terminated = truncated = False
while not (terminated or truncated):
    action = env.action_space.sample()
    obs2, reward, terminated, truncated, info = env.step(action)
    obs = obs2
env.close()
```

Co dostajemy z step?

- obs2: nowa obserwacja/stanu,
- reward: nagroda,
- terminated: koniec MDP,
- truncated: przerwanie przez limit,
- info: diagnostyka.

Co można użyć poza RL?

Planowanie / grafy

BFS, Dijkstra, A*: gdy znamy mapę, koszt przejścia i cel. Świetne dla labiryntów bez ukrytej dynamiki.

Programowanie dynamiczne

Value iteration, policy iteration: gdy znamy $P(s' | s, a)$ i R dla wszystkich stanów.

Uczenie nadzorowane

Gdy mamy pary wejście–etykieta, np. obrazy i klasy. Nie wystarczy, gdy trzeba samemu zebrać doświadczenie i ocenić długofalowy efekt.

Zasada wyboru

Najpierw użyj najprostszego narzędzia, które wykorzystuje dostępną wiedzę: graf/DP, jeśli model jest znany; RL, gdy trzeba uczyć się przez interakcję.

Value Iteration: gdy model jest znany

Input: Zbiory $\mathcal{S}, \mathcal{A}$, model P , nagrody R , dyskonto γ , tolerancja θ

Zainicjalizuj $V(s)$, np. zerami

repeat

$\Delta \leftarrow 0$

foreach $s \in \mathcal{S}$ *nie-terminalny* **do**

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s'} P(s' | s, a)(R(s, a, s') + \gamma V(s'))$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

end

until $\Delta < \theta$

Zwróć $\pi(s) = \arg \max_a \sum_{s'} P(s' | s, a)(R(s, a, s') + \gamma V(s'))$

Cena

Potrzebna jest pełna tablica przejść. W praktyce często jej nie znamy albo jest za duża.

DP vs RL: to samo równanie, inne źródło wiedzy

DP: oczekiwanie po modelu

$$\sum_{s'} P(s' | s, a) (R + \gamma V(s'))$$

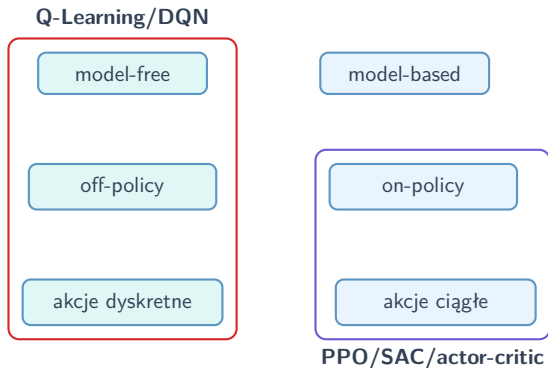
- przeglądamy wszystkie możliwe s' ,
- znamy prawdopodobieństwa,
- zwykle szybkie dla małych MDP.

RL: próbka z doświadczenia

$$(s, a, r, s') \Rightarrow r + \gamma \max_{a'} Q(s', a')$$

- widzimy jeden efekt akcji,
- model może być nieznany,
- uczymy się przez wiele epizodów.

Taksonomia metod RL



Dwa ważne rozróżnienia

model-free (*bez jawnego modelu*): uczymy wartości/polityki z doświadczenia. **off-policy** (*poza-politykowe*): dane mogą pochodzić z polityki eksploracyjnej, a uczymy politykę zachłanną.

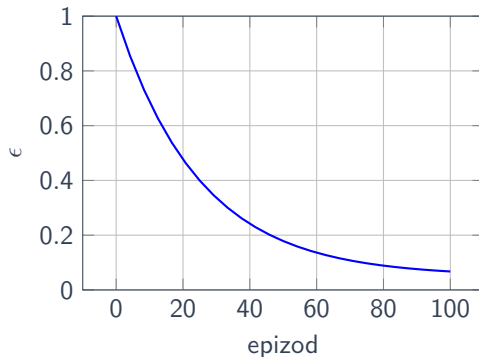
Eksploracja vs eksploatacja

Dylemat

- **eksploatacja** (*exploitation*): wybieram akcję, która wygląda najlepiej.
- **eksploracja** (*exploration*): próbuję niepewnych akcji, aby odkryć lepszą strategię.

ϵ -greedy

$$A_t = \begin{cases} \text{losowa akcja,} & \text{z prawdopodobieństwem } \epsilon, \\ \arg \max_a Q(S_t, a), & \text{z prawdopodobieństwem } 1 - \epsilon \end{cases}$$



Praktyka: dużo eksploracji na początku, stopniowo coraz więcej eksploatacji.

Q-Learning: co zapisuje Q-table?

Tabela jakości

Dla dyskretnych $\mathcal{S}$ i $\mathcal{A}$ przechowujemy macierz:

$$Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$$

Wiersz = stan, kolumna = akcja.

stan	lewo	dół	prawo	góra
0	0.01	0.03	0.72	0.00
1	0.00	0.08	0.64	0.02
2	0.05	0.60	0.10	0.04
3	0.00	0.00	0.00	0.00

Polityka po treningu

$$\pi(s) = \arg \max_a Q(s, a)$$

Reguła aktualizacji

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Target

$$y = r + \gamma \max_{a'} Q(s', a')$$

„Lepsza” ocena po zobaczeniu nowego stanu.

Błąd TD

$$\delta = y - Q(s, a)$$

TD error (*temporal-difference error*)

Krok uczenia

$$Q \leftarrow Q + \alpha \delta$$

α kontroluje, jak mocno ufamy nowej próbie.

Q-Learning: pseudokod w epizodach

```
Input:  $\alpha, \gamma, \epsilon$ , liczba epizodów  
 $Q[s, a] \leftarrow 0$  dla wszystkich  $s, a$   
for  $episode = 1, 2, \dots, N$  do  
     $s \leftarrow env.reset()$   
    while  $s$  nie jest terminalny do  
        Wybierz  $a$  metodą  $\epsilon$ -greedy z  $Q[s, \cdot]$   
        Wykonaj  $a$ : obserwuj  $r, s'$   
         $Q[s, a] \leftarrow Q[s, a] + \alpha(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$   
         $s \leftarrow s'$   
    end  
    opcjonalnie zmniejsz  $\epsilon$   
end  
return  $\pi(s) = \arg \max_a Q[s, a]$ 
```

FrozenLake: szkic implementacji Q-Learning

```
import gymnasium as gym
import numpy as np

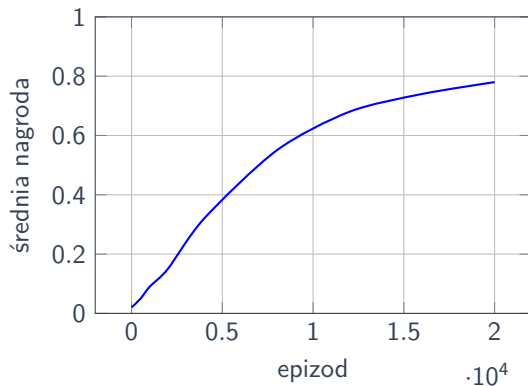
env = gym.make("FrozenLake-v1", is_slippery=True)
nS, nA = env.observation_space.n, env.action_space.n
Q = np.zeros((nS, nA))
alpha, gamma = 0.8, 0.95
eps, eps_min, eps_decay = 1.0, 0.05, 0.995

for episode in range(20_000):
    s, info = env.reset()
    done = False
    while not done:
        if np.random.random() < eps:
            a = env.action_space.sample()
        else:
            a = int(np.argmax(Q[s]))
        s2, r, terminated, truncated, info = env.step(a)
        done = terminated or truncated
        Q[s, a] += alpha * (r + gamma*np.max(Q[s2]) - Q[s, a])
        s = s2
    eps = max(eps_min, eps*eps_decay)
```

Jak mierzyć, czy agent się uczy?

Metryki

- średnia nagroda z ostatnich K epizodów,
- odsetek sukcesów (*success rate*),
- długość epizodu,
- stabilność polityki,
- liczba odwiedzonych stanów.



Nie oceniaj po jednym epizodzie

W środowiskach stochastycznych sukces lub porażka pojedynczego przebiegu może być przypadkiem.

Rzadka nagroda

Agent długo widzi same zera, więc nie wie, które ścieżki są dobre.

Start z zerami

Wszystkie akcje wyglądają tak samo. Decydują losowania i eksploracja.

Poślizg

Najlepsza akcja w sensie deterministycznym może być ryzykowna, jeśli prowadzi obok dziury.

Dobór hiperparametrów

Zbyt szybki spadek ϵ zamyka agenta w słabej polityce. Zbyt duże α może powodować niestabilność.

Dlaczego klasyczna sieć nadzorowana nie wystarczy?

Uczenie nadzorowane

Potrzebujemy danych:

$$(x_i, y_i) = (\text{obserwacja}, \text{poprawna odpowiedź})$$

Problem w RL

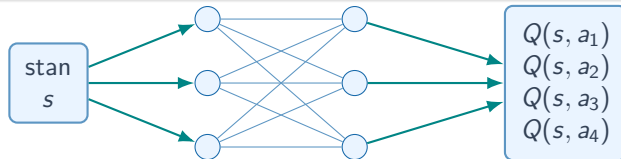
W stanie s nie znamy „poprawnej akcji”. Wiemy dopiero później, czy cała sekwencja decyzji była dobra.



Brakuje etykiety.
Zamiast tego
tworzymy cel
treningowy
z równania
Bellmana.

Problem skali

Jeśli stan jest obrazem, wektorem sensorycznym albo ciągłym zestawem liczb, tablica $|\mathcal{S}| \times |\mathcal{A}|$ przestaje mieć sens.



DQN (*Deep Q-Network*): $Q_{\theta}(s, a)$ zamiast tabeli

Target Bellmanowski

$$y = r + \gamma \max_{a'} Q_{\theta-}(s', a')$$

Uczenie sieci

$$L(\theta) = (y - Q_{\theta}(s, a))^2$$

- Q_{θ} - aktualna sieć.
- $Q_{\theta-}$ - **target network** (*sieć docelowa*), aktualizowana wolniej.
- Nadal potrzebujemy dyskretnych akcji, bo liczymy $\max_{a'}$ po skończonym zbiorze.

DQN: replay buffer i target network

Replay buffer

Przechowuje przejścia:

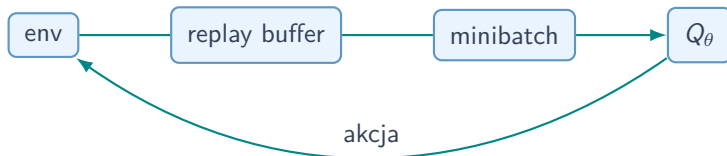
$$(s, a, r, s', done)$$

Trenujemy na losowych minibatchach, żeby ograniczyć korelacje kolejnych obserwacji.

Target network

Kopia sieci generuje stabilniejsze cele y . Co pewien czas:

$$\theta^- \leftarrow \theta$$



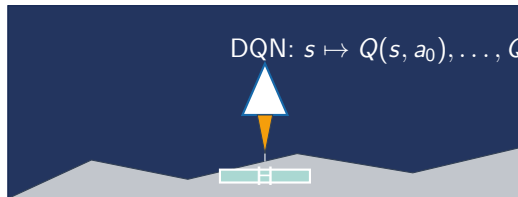
LunarLander-v2: przykład z dyskretnymi akcjami

Obserwacja - wektor 8 liczb

- pozycja (x, y) ,
- prędkości (v_x, v_y) ,
- kąt θ i prędkość kątowna ω ,
- kontakt lewej i prawej nogi z podłożem.

Akcje dyskretnie

- ❶ 0: nic nie rób,
- ❷ 1: lewy silnik,
- ❸ 2: główny silnik,
- ❹ 3: prawy silnik.



Ciągła przestrzeń akcji: dlaczego DQN nie wystarcza?

DQN zakłada

$$\max_{a' \in \{0,1,2,3\}} Q(s', a')$$

Da się policzyć, bo akcji jest mało.

Akcje ciągłe

$$a \in [-1, 1]^d$$

Nie da się przetestować wszystkich możliwych wartości silnika, kąta, momentu itp.

LunarLanderContinuous-v2

Akcja może oznaczać ciągłą siłę głównego silnika i ciągłą kontrolę silników bocznych, np. wektor rzeczywisty zamiast numeru akcji.

Actor-Critic: standardowa odpowiedź na akcje ciągłe

Actor (*aktor*)

Sieć reprezentująca politykę:

$$\pi_{\theta}(a | s) \quad \text{albo} \quad a = \mu_{\theta}(s)$$

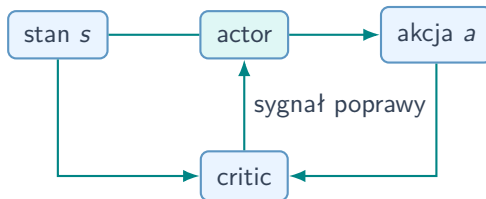
Generuje akcję, np. wartość przepustnicy.

Critic (*krytyk*)

Sieć oceniająca:

$$V_{\phi}(s) \quad \text{lub} \quad Q_{\phi}(s, a)$$

Mówi aktorowi, czy decyzja była dobra.



Dobór algorytmu do typu problemu

Sytuacja	Model znany?	Akcje	Przykładowe metody
Mały MDP, pełny model	tak	dyskretne	value iteration, policy iteration
Mały MDP, model nie- znany	nie	dyskretne	Q-Learning, SARSA
Duży/ciągły stan	nie	dyskretne	DQN, Double DQN, Dueling DQN
Duży/ciągły stan	nie	ciągłe	DDPG, TD3, SAC, PPO
Znana mapa i koszty	tak	ruchy w grafie	BFS, Dijkstra, A*

Krytyczne pytanie projektowe

Czy możemy tanio i bezpiecznie zbierać doświadczenie? Jeśli nie, RL może być zbyt kosztowne albo ryzykowne.

Mini-projekt dla studentów

- 1 Uruchom FrozenLake-v1 w wariancie deterministic i slippery.
- 2 Zaimplementuj Q-Learning z ϵ -greedy.
- 3 Porównaj wpływ α , γ , harmonogramu ϵ oraz liczby epizodów.
- 4 Zmierz *success rate* na 1000 epizodów bez eksploracji.
- 5 Zastanów się: kiedy value iteration byłby prostszy?

Rozszerzenie

Porównaj Q-Learning na FrozenLake z DQN na LunarLander-v2. Nie oceniaj wyłącznie po wyniku - pokaż krzywą uczenia.

- RL jest naturalne dla sekwencyjnych decyzji z opóźnioną nagrodą.
- MDP porządkuje pojęcia: stan, akcja, model, nagroda, polityka, dyskonto.
- DP jest mocne, gdy znamy model; RL jest potrzebne, gdy model poznajemy z doświadczenia.
- Q-Learning uczy $Q(s, a)$ z próbek i nie wymaga modelu.
- DQN zastępuje Q-table siecią, ale nadal zakłada dyskretne akcje.
- Dla akcji ciągłych przechodzimy zwykle do metod actor-critic.

- 1 Czym różni się model $P(s' | s, a)$ od polityki $\pi(a | s)$?
- 2 Dlaczego γ pomaga przy nieskończonym horyzoncie?
- 3 Co oznacza błąd TD w Q-Learning?
- 4 Dlaczego Q-Learning jest off-policy?
- 5 Dlaczego DQN nie rozwiązuje bezpośrednio akcji ciągłych?
- 6 W jakiej sytuacji użyłbyś A^* zamiast RL?

- Materiały z wykładu o MDP i Gym/Gymnasium: definicje agenta, środowiska, stanów, akcji, przejść, polityki oraz przykłady LunarLander i FrozenLake.
- Materiały z wykładu o RL: zwroty, dyskonto, funkcje V i Q , równania Bellmana, value iteration, Q-Learning i DQN.
- Dokumentacja Gymnasium: środowiska FrozenLake-v1, LunarLander-v2, LunarLanderContinuous-v2.

Dziękuję!